



Auteur : [LOUVEL Liam](#)

Type : Lab cybersécurité fictif / environnement de test

Version doc : 1.0

Date : 2026-05-10

SOMMAIRE :

1. Contexte et objectifs
2. Environnement de test
3. Méthodologie
4. Résultats - Reconnaissance
5. Résultats - Brute Force
6. Résultats - XSS
7. Résultats - Interception HTTP (Burp Suite)
8. Résultats - Capture réseau (Wireshark)
9. Tableau récapitulatif des vulnérabilités
10. Conclusion et recommandations



Note

Ce rapport est mon premier livrable red team, réalisé dans le cadre d'un apprentissage personnel sur environnement isolé.



Ce document a été rédigé avec l'assistance de l'IA Claude (Anthropic). Les tests, manipulations et découvertes techniques sont le travail de l'auteur. La rédaction a été assistée par IA en raison d'une dysorthographe.



Avertissement

Ce rapport documente des tests effectués sur un environnement fictif et isolé (lab personnel). Toutes les techniques présentées ici sont **ILLÉGALES** sans autorisation écrite sur des systèmes réels (Article 323-1 du Code Pénal - jusqu'à 2 ans de prison et 60 000€ d'amende).

1. CONTEXTE ET OBJECTIFS :

Ce rapport documente la phase Red Team v1 du projet NEXORA.

L'objectif est de tester la sécurité de l'infrastructure mise en place lors de la Blue Team v1 en simulant les actions d'un attaquant réel.

(objectif apprentissage des outils et méthode de la Red Team)

Résumé exécutif :

L'environnement NEXORA v1 a été compromis avec succès depuis un accès réseau interne standard. La phase de reconnaissance a permis d'identifier plusieurs informations sensibles exposées publiquement, notamment les versions exactes d'Apache et d'OpenSSH ainsi qu'un formulaire d'authentification accessible sur le serveur web.

L'absence de protection contre les attaques par force brute, combinée à l'utilisation d'un mot de passe faible, a permis la compromission du compte administrateur en moins de 30 secondes via Hydra. Les échanges HTTP non chiffrés ont ensuite permis l'interception des identifiants administrateur en clair à l'aide de Burp Suite et Wireshark.

Des vulnérabilités XSS réfléchies ont également été identifiées sur le formulaire de connexion, permettant l'exécution de code JavaScript arbitraire côté client.

Au total, 15 vulnérabilités ont été identifiées, dont 4 critiques, démontrant qu'un attaquant disposant d'un accès au réseau local peut compromettre rapidement l'infrastructure sans techniques avancées.

Périmètre du test :

- Serveur web Ubuntu (192.168.10.10)
- Application web login PHP (/index.php)
- Services exposés : HTTP (80), SSH (22)

Outils utilisés :

- nmap 7.98
- hydra v9.6
- Burp Suite
- Wireshark
- Firefox

2. ENVIRONNEMENT DE TEST :

Rôle	OS	IP
Cible (serveur)	Ubuntu Server	192.168.10.10
Attaquant	Kali Linux	192.168.10.20

Réseau interne : NEXORA (VirtualBox)

Protocole testé : HTTP (pas de HTTPS)

3. MÉTHODOLOGIE :

Les tests ont suivi la méthodologie classique d'un pentest :

Étape 1 : Reconnaissance passive et active (nmap)

Étape 2 : Attaque par force brute (hydra)

Étape 3 : Test d'injection XSS (Firefox)

Étape 4 : Interception de trafic (Burp Suite + Wireshark)

Aucun outil automatisé d'exploitation n'a été utilisé autre que de ceux listés ci-dessus.

4. RÉSULTATS - RECONNAISSANCE :

OUTIL : nmap 7.98

----- Scan basique -----

Commande :

nmap 192.168.10.10

Résultat :

PORT STATE SERVICE

22/tcp open ssh

80/tcp open http

MAC Address: 08:00:27:0A:CA:B6 (Oracle VirtualBox virtual NIC)

-----Scan détaillé -----

Commande :

nmap -sV -A 192.168.10.10

Résultat :

PORT STATE SERVICE VERSION

22/tcp open ssh OpenSSH 10.2p1 Ubuntu 2ubuntu3.2

80/tcp open http Apache httpd 2.4.66 ((Ubuntu))

http-title: NEXORA Login

http-server-header: Apache/2.4.66 (Ubuntu)

OS détecté : Linux

VULNÉRABILITÉS IDENTIFIÉES :

Vulnérabilité 1 : Version exacte d'OpenSSH exposée (10.2p1)

→ Permet de cibler des CVE spécifiques à cette version

Vulnérabilité 2 : Version exacte d'Apache exposée (2.4.66)

→ Idem, visible dans chaque réponse HTTP

Vulnérabilité 3 : Titre de page révèle l'existence d'un login

→ "NEXORA Login" confirme une surface d'attaque

Vulnérabilité 4 : Identification de la virtualisation VirtualBox

→ Révèle un environnement de test potentiel

CRITICITÉ : Faible (information disclosure)

Mais facilite grandement les attaques suivantes.

5. RÉSULTATS - BRUTE FORCE :

OUTIL : hydra v9.6

CIBLE : Formulaire POST /index.php

Commande :

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt
```

```
192.168.10.10
```

```
http-post-form
```

```
"/index.php:username=^USER^&password=^PASS^:Invalid credentials"
```

Wordlist utilisée : rockyou.txt (14 344 399 mots de passe)

Résultat :

```
[80][http-post-form] host: 192.168.10.10
```

```
login: admin password: password123
```

```
1 of 1 target successfully completed, 1 valid password found
```

Durée : 22 secondes

Tentatives : ~350 avant de trouver le bon mot de passe

VULNÉRABILITÉS IDENTIFIÉES :

Vulnérabilité 5 : Aucune limite de tentatives de connexion

→ hydra a testé des centaines de mots de passe

sans être bloqué ni ralenti

Vulnérabilité 6 : Mot de passe faible (password123)

→ Présent dans les 400 premiers de rockyou.txt

Vulnérabilité 7 : Pas de délai entre les tentatives

→ Le serveur répond instantanément à chaque essai

Vulnérabilité 8 : Pas de CAPTCHA

→ Aucune différenciation humain/robot

CRITICITÉ : CRITIQUE

Credentials admin compromis en 22 secondes.

6. RÉSULTATS - XSS (CROSS-SITE SCRIPTING) :

OUTIL : Firefox

CIBLE : Champ username du formulaire /index.php

TYPE : XSS réfléchi (reflected)

Contexte :

Le code PHP affiche le username dans le message d'erreur
sans filtrage :

```
$message = "Invalid credentials for user : " . $user;
```

```
<h2><?php echo $message; ?></h2>
```

Payload injecté :

```
<script>alert('XSS BLUME')</script>
```


Résultat :

- Pop-up JavaScript exécuté dans le navigateur
- Confirmation que le code JS est interprété par le navigateur

Payload de démonstration défiguration :

```
<script>document.body.innerHTML='<h1>HACKED</h1>'\</script>
```

- Page entière remplacée côté client

VULNÉRABILITÉS IDENTIFIÉES :

Vulnérabilité 9 : Absence de htmlspecialchars() sur les variables affichées

- Tout input utilisateur est injecté directement dans le HTML

Vulnérabilité 10 : Pas de Content Security Policy (CSP)

- Aucune restriction sur l'exécution de scripts

CRITICITÉ : ÉLEVÉE

En XSS réfléchi POST : impact limité à l'attaquant lui-même.

En XSS stocké (si base de données) : impact sur tous les visiteurs (vol de session, défiguration, exfiltration).

NOTE : Le formulaire utilisant POST et non GET, le scénario de lien piégé envoyé à une victime est limité dans ce contexte.

7. RÉSULTATS - INTERCEPTION HTTP (BURP SUITE):

OUTIL : Burp Suite (proxy 127.0.0.1:8080)

CIBLE : Requête POST /index.php

Configuration :

Firefox configuré avec proxy manuel → 127.0.0.1:8080

Burp Suite en mode Intercept ON

Requête interceptée :

POST / HTTP/1.1

Host: 192.168.10.10

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0)

Content-Type: application/x-www-form-urlencoded

Content-Length: 35

username=admin&password=password123

VULNÉRABILITÉS IDENTIFIÉES :

Vulnérabilité 11 : Credentials transmis en clair (HTTP sans TLS)

→ username=admin&password=password123 lisible

directement dans la requête POST

Vulnérabilité 12 : Absence de token CSRF

→ Pas de protection contre les requêtes forgées

Vulnérabilité 13 : Header "Server" expose la version Apache

→ Server: Apache/2.4.66 (Ubuntu) dans chaque réponse

CRITICITÉ : **CRITIQUE**

Tout proxy ou attaquant sur le réseau voit les credentials en clair sans aucun effort.

8. RÉSULTATS - CAPTURE RÉSEAU (WIRESHARK) :

OUTIL : Wireshark

INTERFACE : eth1 (réseau NEXORA 192.168.10.20)

FILTRE : http

Flux HTTP complet capturé (Follow HTTP Stream) :

REQUÊTE :

POST / HTTP/1.1

Host: 192.168.10.10

Content-Type: application/x-www-form-urlencoded

username=admin&password=password123

RÉPONSE :

HTTP/1.1 200 OK

Server: Apache/2.4.66 (Ubuntu)

<h2 class="message">welcom admin!</h2>

Informations extraites en une seule capture :

- Credentials admin en clair (username + password)
- Version exacte d'Apache
- Code source HTML complet de la page
- Confirmation du login réussi ("welcom admin!")
- Structure complète du formulaire

VULNÉRABILITÉS IDENTIFIÉES :

Vulnérabilité 14 : Trafic HTTP non chiffré interceptable par tout équipement sur le réseau (attaque Man-in-the-Middle)

Vulnérabilité 15 : Code source de l'application visible dans le trafic

→ Structure, CSS, logique frontend exposés

CRITICITÉ : **CRITIQUE**

Un attaquant passif (sans même interagir avec le serveur) peut capturer les credentials de n'importe quel utilisateur qui se connecte sur le même réseau.

9. TABLEAU RÉCAPITULATIF DES VULNÉRABILITÉS :

ID	Vulnérabilité	Outil	Criticité
V1	Version OpenSSH exposée	nmap	Faible
V2	Version Apache exposée	nmap	Faible
V3	Titre page révèle login	nmap	Faible
V4	VirtualBox identifié	nmap	Info
V5	Pas de limite de tentatives	hydra	Critique
V6	Mot de passe faible	hydra	Critique
V7	Pas de délai entre tentatives	hydra	Élevée
V8	Pas de CAPTCHA	hydra	Élevée
V9	Absence htmlspecialchars()	Firefox	Élevée
V10	Pas de CSP	Firefox	Élevée
V11	Credentials en clair (HTTP)	Burp	Critique
V12	Absence token CSRF	Burp	Élevée
V13	Header Server expose version	Burp	Faible
V14	Trafic interceptable (MitM)	Wireshark	Critique
V15	Code source visible réseau	Wireshark	Faible

Bilan :

Critique : 4 vulnérabilités

Élevée : 6 vulnérabilités

Faible : 4 vulnérabilités

Info : 1 vulnérabilité

10. CONCLUSION ET RECOMMANDATIONS :

CONCLUSION :

L'infrastructure NEXORAv1 présente de nombreuses vulnérabilités **critiques**. Un attaquant disposant d'un accès au réseau interne peut compromettre le compte admin en moins de **30secondes** sans **compétences avancées**.

Les trois vecteurs d'attaque les plus critiques sont :

1. Brute force sans protection → credentials en 22 secondes
2. HTTP sans chiffrement → credentials visibles sur le réseau
3. XSS réfléchi → exécution de code JS arbitraire

RECOMMANDATIONS POUR BLUE TEAM v2 :

Recommandation 1 : Mettre en place HTTPS (certificat auto-signé)

→ Chiffre le trafic, rend Wireshark/Burp inutiles sur les credentials

Recommandation 2 : Hasher les mots de passe

→ password_hash() / password_verify() en PHP

Recommandation 3 : Limiter les tentatives de connexion

→ Bloquer une IP après X tentatives échouées (entre 3 et 5)

Recommandation 4 : Ajouter htmlspecialchars() sur toutes les variables

affichées → corrige le XSS

Recommandation 5 : Masquer la version Apache

→ ServerTokens Prod dans apache2.conf

Recommandation 6 : Ajouter un token CSRF sur le formulaire

Recommandation 7 : Mettre en place un firewall ufw

→ Limiter les ports exposés

Recommandation 8 : Politique de mot de passe forte

→ Longueur minimale, complexité, pas dans rockyou.txt

**FIN DU RAPPORT - NEXORA Red
Team v1**

merci d'avoir lu !